

Nombre de la práctica	Problemario DDL y Constraints en MySQL.			No.	3
Asignatura:	Taller de Base de Datos	Carrera:	ISIC	Duración de la práctica (Hrs)	4

I. Competencia(s) específica(s): Realizar bases de datos con constraints

II. Lugar de realización de la práctica (laboratorio, taller, aula u otro): Aula

III. Material empleado:

Computadora

IV. Desarrollo de la práctica:

Problema 1: Registro de empleados con restricciones salariales.

Una empresa quiere guardar los Empleados, no cuentan con un registro como numero de empleados, anteriormente se tenia registro en un Excel con el nombre, email y salario, las reglas de la empresa no permiten un salario mensual menor a \$3000 ni mayor a \$50000. Crea la base de datos llamada Ejercicio_Constraints_1 y la tabla Empleados que cumpla con estos requisitos.



empleados	
	id_empleado : int
	nombre : varchar(255)
	apellido1 : varchar(255)
	apellido2 : varchar(255)
	email : varchar(255)
	salario : int

```
CREATE TABLE Empleados(
id_empleado INT PRIMARY KEY AUTO_INCREMENT,
nombre VARCHAR(255) NOT NULL,
apellido1 VARCHAR(255) NOT NULL,
apellido2 VARCHAR(255) NOT NULL,
email VARCHAR(255) NOT NULL,
salario INT CHECK(salario > 3000 & salario < 50000)
);
```

1.- En este ejercicio el constraint utilizado es para salario en el cual no puede ser menor a 3000 ni mayor a 50000

Problema 2: Relación entre empleados y departamentos

Una empresa necesita organizar a sus empleados según los departamentos a los que pertenecen. Cada departamento tiene un nombre único. La empresa quiere almacenar la información de los empleados junto con el departamento al que están asignados. Actualmente, cada empleado tiene un número de identificación. Se requiere crear una relación entre ambas tablas para que cada empleado esté asignado a un departamento.

Crea la base de datos llamada `Ejercicio_Constraints_2` y las tablas `Empleados` y `Departamentos` con las restricciones necesarias para cumplir con esta relación.



```
CREATE TABLE Empleados (
    numero_de_empleado INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(255),
    FOREIGN KEY (nombre) REFERENCES departamentos(nombre)
);
```

```
CREATE TABLE Departamentos (
    id_departamento INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(255) UNIQUE NOT NULL
);
```

2.- En este ejercicio utilizamos un `UNIQUE` ya que el ejercicio nos pide que el nombre del departamento sea único.

Problema 3: Control de inventario de productos Una tienda quiere llevar el control de los productos que vende.

La información almacenada en la tabla de `Productos` debe incluir el nombre del producto, código de barras y su precio. A su vez, necesitan añadir una columna para el stock de cada producto, la cual no puede ser nula y debe tener un valor por defecto de 100. Además, el precio de los productos debe ser siempre mayor a 0, y el nombre de cada producto debe ser único para evitar duplicados.

Crea la base de datos llamada `Ejercicio_Constraints_3` y realiza las modificaciones necesarias en la tabla `Productos` para cumplir con estos requisitos.



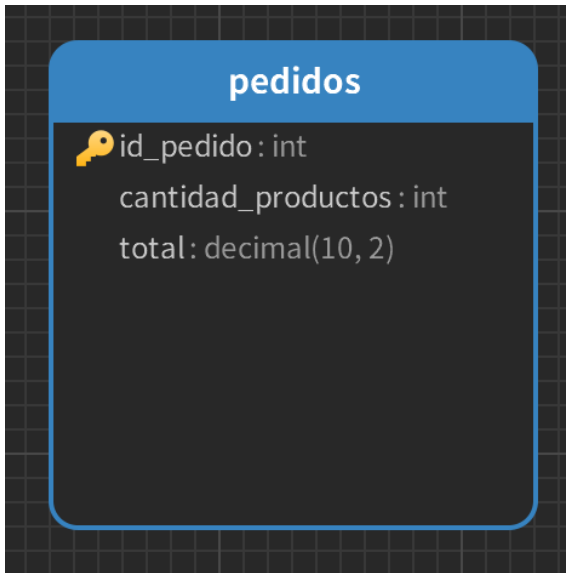
```
CREATE TABLE Productos (
  id_producto INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(255) UNIQUE NOT NULL,
  codigo_barras VARCHAR(255) NOT NULL,
  precio DECIMAL(10, 2) CHECK (precio > 0) NOT NULL,
  stock INT DEFAULT 100 NOT NULL
);
```

3.- En el ejercicio 3 utilizamos un check para el precio ya que no puede ser igual o menor a 0, un unique para nombre único del producto

Problema 4: Control de pedidos con validación de montos

Una empresa quiere registrar los pedidos que recibe, pero necesita asegurarse de que el total de cada pedido sea proporcional a la cantidad de productos solicitados. Específicamente, el total de cada pedido debe ser al menos igual a la cantidad de productos multiplicada por 10.

Además, cada pedido debe contener al menos 1 producto. Crea la base de datos llamada Ejercicio_Constraints_4 y la tabla Pedidos que cumpla con estas validaciones usando restricciones CHECK



```
CREATE TABLE Pedidos (
  id_pedido INT AUTO_INCREMENT PRIMARY KEY,
  cantidad_productos INT NOT NULL CHECK (cantidad_productos >= 1),
  total DECIMAL(10, 2) NOT NULL,
  CHECK (total >= cantidad_productos * 10)
);
```

4.- En este ejercicio utilizamos dos check uno para que la cantidad del producto mínima sea 1 y y el pago del producto sea la cantidad de ellos * 10 minimo

Problema 5: Control de ventas de productos por empleados

Una empresa de ventas necesita registrar las ventas que realiza. Cada venta está asociada a un empleado y a un producto específico. Para garantizar la integridad de los datos, se requiere que cada venta tenga la referencia tanto del empleado como del producto, y que las ventas sean realizadas en una fecha válida (no futura).

Además, la cantidad de productos vendidos debe ser mayor a 0. Crea la base de datos llamada Ejercicio_Constraints_5 y las tablas necesarias para almacenar esta información, incluyendo las claves foráneas para relacionar las tablas de empleados y productos con las ventas.



```
CREATE TABLE Empleados (  
    id_empleado INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE Productos (  
    id_productos INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(255) NOT NULL,  
    precio DECIMAL(10, 2) NOT NULL CHECK (precio > 0)  
);
```

```
CREATE TABLE Venta (  
    id_venta INT AUTO_INCREMENT PRIMARY KEY,  
    id_empleado INT,  
    id_productos INT,  
    FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE  
ON DELETE RESTRICT,  
    FOREIGN KEY (id_productos) REFERENCES productos(id_productos) ON UPDATE CASCADE  
ON DELETE RESTRICT,  
    fecha DATE CHECK (fecha <= '2024-09-29')  
);
```

5.- En este ultimo utilizamos dos check uno para que el precio del producto sea mayor a 0 y otro para que la fecha sea valida y no mayor a la de hoy

V. Conclusiones:

En esta practica entendí la forma en la que podemos utilizar constraints para que en la realización de bases de datos no tengamos problemas por ejemplo cuando una nota lleva una fecha debe de ser la de hoy o mínimo no debe de ser futura ya que si esa nota ya esta impresa debe ser de ese mismo día o días antes, alertar que una venta de productos puede estar mal si la cantidad de productos multiplicada por no 10 no llega a cubrir esta cantidad no se permita entre otras funcionalidades importantes que nos pueden evitar el tener problemas a futuro.